

等待策略

现代 Web 页面是异步渲染的，元素什么时候出现、什么时候可交互，都不确定。****等待策略是自动化测试稳定性的核心****，写错了就会得到时好时坏的案例。

四种等待策略

NONE

不等待，立刻执行。

只用在****确定元素已经就绪****的场景，比如上一步刚断言过它可见。绝大多数步骤不该用 `NONE`。

PRESENCE

等待元素出现在 DOM 中，****不要求可见****。

用于操作那些故意隐藏的元素，最典型的是文件上传 —— `<input type="file">` 通常被 CSS 藏起来，用 `VISIBLE` 会永远等不到。

VISIBLE

等待元素出现且可见（在 DOM 中 + 没有 `display:none` / `visibility:hidden` + 宽高大于 0）。

****所有断言类 action 都应该用 `VISIBLE`。断言一个不可见的元素没有意义 —— 用户看不到的东西，验证它做什么。**

CLICKABLE

等待元素可见****且可交互****（没有被 `disabled`、没有被其他元素遮挡）。

****所有 `CLICK` 步骤必须用 `CLICKABLE`。****

seq	4
action	CLICK
locator_type	XPATH
locator_value	//button[@data-testid="login-submit"]
wait_strategy	CLICKABLE 00000STD-0050
wait_timeout_sec	10
on_failure	ABORT

图：案例编辑页里 CLICK 步骤的 wait_strategy 字段，显示 CLICKABLE 由平台自动补完

上图是案例编辑页的步骤表单。注意 `wait\_strategy` 这一栏——选了 `CLICK` 之后它会被平台自动填成 `CLICKABLE` 并置灰，不需要也不能手工改。

为什么 CLICK 必须用 CLICKABLE

可见不等于可点

这是新人最难理解、也最容易踩的一点。一个元素可见，但可能：

- 带着 `disabled` 属性（表单校验没通过，提交按钮是灰的）
- 被遮挡（有个加载遮罩盖在上面，或者悬浮的客服按钮挡住了）
- 事件还没绑定（DOM 渲染完了，但框架的事件注册在下一个 tick 才执行）

前两种情况点击会直接报错，第三种最阴险：**点击「成功」了，但什么也没发生**——因为那一刻还没有监听器来处理这个事件。

这类失败为什么难查

用 `VISIBLE` 点击导致的失败有三个特征，凑在一起极其难查：

1. **偶发** —— 依赖机器负载和网络速度，本地跑十次都好的，CI 上就挂
2. **报错位置不在真正的问题处** —— 点击那一步「成功」了，失败发生在后面某个断言上，说「元素找不到」
3. **加个 SLEEP 就好了** —— 于是很多人就真的加了 `SLEEP`，问题被掩盖，规范被违反

所以 ATP 在保存案例时**自动把 `CLICK` 的 `wait\_strategy` 强制填成 `CLICKABLE`**，不给人写错的机会。同理，`ASSERT\_\*` 会被自动填成 `VISIBLE`。

为什么禁用 SLEEP

固定等待的两难

`SLEEP` 的问题不是「不好用」，而是它**必然在两个坏结果里二选一**：

- 等的时间**长于**实际需要 每条案例都白白浪费时间，几百条案例的回归就是几十分钟
- 等的时间**短于**实际需要 偶发失败，而且会随着环境变慢而越来越频繁

不存在一个「刚刚好」的值，因为实际需要的时间本身就是变化的。

它掩盖真正的问题

更严重的是，`SLEEP` 让人止步于「能跑通了」。

一个步骤加了 `SLEEP 3` 之后通过了，真实情况往往是等待策略写错了——比如该用 `CLICKABLE` 的用了 `VISIBLE`。`SLEEP` 恰好盖住了这个时间差，于是根因永远没被发现，直到某天环境变慢，`SLEEP 3` 不够了，同样的问题以「加到 5 秒」的形式再来一次。

所以规范禁用 `SLEEP` 不只是为了执行速度，更是为了**逼人去找真正的原因**。详细论证见《待機戰略規約》。

历史案例怎么办

平台里还存在少量使用 `SLEEP` 的历史案例，这些是规范建立之前留下的。

处理原则：**不主动批量改，但碰到就顺手修**。如果你因为别的原因要动一条含 `SLEEP` 的案例，顺便把它改成显式等待。批量改动的风险大于收益。

超时时间怎么设

两个层级的超时

容易混淆的两个字段：

字段	作用范围	默认	范围
---	---	---	---
`tc_case.timeout_sec`	**整条案例**	30	5~300
`tc_step.wait_timeout_sec`	**单个步骤**	10	1~120

案例级超时是总闸——即使每个步骤都没超时，累计时间超了整条也会被终止。

所以设置时要保证：**所有步骤的等待时间之和留有余量地小于案例超时**。一条有 10 个步骤、每步 10 秒等待的案例，理论最坏耗时 100 秒，案例超时如果还是默认的 30 秒，就会在跑到一半时被砍掉。

不要盲目调大

遇到超时失败，第一反应不应该是调大超时。

先判断是哪种情况：

- ****元素最终会出现，只是慢**** 可以适当调大，但要想想为什么慢，是不是被测系统有性能问题
- ****元素根本不会出现**** 调多大都没用，是定位器错了或者页面状态不对
- ****元素出现了但不满足等待条件**** 比如用了 ``CLICKABLE`` 但按钮一直是 disabled，说明前置操作没做对

盲目调大超时的代价是：真正该失败的案例要等满超时才报错，一条几十秒，几百条就是很长的等待。

****而且它同样掩盖问题**** —— 这一点和 ``SLEEP`` 是同一个道理。

一个典型的错误案例

下面这个步骤序列看起来很合理，但有两个问题：

```
| seq | action | wait_strategy | 问题 |  
|---|---|---|---|  
| 1 | `CLICK` 提交 | `VISIBLE` | 应该是 `CLICKABLE` |  
| 2 | `SLEEP` 3 秒 | — | 禁止使用 |  
| 3 | `ASSERT_TEXT` 成功提示 | `VISIBLE` | |
```

正确的写法是把第 1 步改成 ``CLICKABLE``，然后****直接删掉第 2 步**** —— 第 3 步的 ``ASSERT_TEXT`` 自带 ``VISIBLE`` 等待，它会一直等到成功提示出现为止，最多等 ``wait_timeout_sec`` 秒。

这是最常见的改造模式：****大多数 `SLEEP` 删掉就行，不需要用别的东西替代****，因为下一步的等待已经覆盖了它。